DIVIDE Y VENCERÁS



Antonio Miguel Almagro Valles
Javier Díaz Vilchez
Rodrigo láñez Huertas

1.Objetivo

Se nos pide construir soluciones utilizando el método algorítmico divide y vencerás, que consiste en dividir el problema en subproblemas, resolver esos subproblemas y luego combinar los resultados para obtener la solución final del problema.

Debemos comparar los algoritmos implementados tanto cualitativa como cuantitativamente y realizar un análisis de la eficiencia de las soluciones aportadas, mostrando una comparativa desde el punto de vista teórico y práctico.

El problema que debemos resolver es organizar un torneo de tenis, donde nos encargaremos de desarrollar el cuadro de cruces del torneo/liga de todos contra todos.

Jug/día	d1	d2	d3	d4	d5
1	2	3	4	5	--
2	1	5	3	--	4
3	--	1	2	4	5
4	5	--	1	3	2
5	4	2	--	1	3

Jug/día	d1	d2	d3	d4	d5
1	2	3	4	5	6
2	1	5	3	6	4
3	6	1	2	4	5
4	5	6	1	3	2
5	4	2	6	1	3
6	3	4	5	2	1

Sabemos que tenemos n participantes en donde cada jugador ha de jugar exactamente una vez contra cada uno de sus posibles n-1 competidores, y además debe jugar un partido cada día, teniendo a lo sumo un día de descanso en todo el torneo.

Teniendo en cuenta lo anterior, consideremos los siguiente tres casos:

1. Si n es potencia de 2, implementaremos un algoritmo para construir un cuadro de partidos del torneo en **n - 1 días** y con **n jugadores.**

2. Dado cualquier n>1, implementaremos un algoritmo para construir un cuadro de partidos del torneo que permita terminarlo en **n -1 días** si n es **par**, o en **n días** si n es **impar**.

3. Si n es potencia de 2 (siendo esta vez n el número de días), al igual que en el primer caso, implementaremos un algoritmo para construir un cuadro de partidos del torneo con **n - 1 jugadores** y **n días**.

Para el desarrollo de los algoritmos que se nos han propuesto, nos hemos dividido las tareas, cada uno hemos tenido nuestra parte en la realización de estos y después del trabajo individual, una puesta en común para revisar y mejorar las propuestas.

El desglose de las actividades a realizar ha sido el siguiente:

Para la implementación del Caso 1: Javier y Rodrigo
Para el caso 2.1: Javier, Rodrigo y Antonio
Para el caso 2.2: Javier, Rodrigo y Antonio
Para el caso 3: Antonio
Test: Javier, Rodrigo y Antonio
Pruebas de rendimiento: Rodrigo y Antonio
Estudio de implementación: Javier
Estudio teórico: Rodrigo
Estudio experimental: Antonio

Como hemos comentado anteriormente, al final todos hemos puesto nuestra parte en cada punto, el desglose anterior se nombra a las personas principales encargadas de dicho apartado.

2.Estudio de implementación

Para los 3 casos, trabajando con un algoritmo de “Divide y Vencerás”, necesitaremos crear un algoritmo que sea recursivo. Para los 3 algoritmos partimos de un caso base común de orden constante para n = 2 días. Para las tablas creadas, no consideramos nunca las posiciones correspondientes a la fila 0 y la columna 0, ya que no serían válidos el día 0 y el jugador 0. Partiendo de esto, para el caso base, el jugador 1 jugaría contra el jugador 2 y viceversa, por lo que determinamos directamente los valores para las posiciones (1,1) y (2,1) en las tablas. Dicho esto, continuamos con las explicaciones correspondientes a cada caso (las implementaciones en pseudocódigo con comentarios breves):

2.1. Caso 1: n es potencia de dos

Para el caso 1, hemos definido dos métodos: el método torneoRecursivoCaso1, el cual va a tratar el caso base y las llamadas recursivas, y el método completarTablaCaso1 que va a completar los diferentes cuadrantes de la tabla. Su implementación es la siguiente:

Método TorneoRecursivoCaso1

```
Método torneoRecursivoCaso1(tabla, n días)
    Si n es igual a 2 entonces //caso base
        tabla[1][1] = 2
        tabla[2][1] = 1
    Sino
```

torneoRecursivoCaso1(tabla, n/2) ***//*llamadas recursivas**
completarTablaCaso1(tabla, n)
Fin del si
Fin

Método CompletarTablaCaso1
Método privado completarTablaCaso1(tabla, n días)
medio = n/2
Para i desde medio + 1 hasta n
Para j desde 1 hasta medio - 1
tabla[i][j] = tabla[i - medio][j] + medio
Fin del bucle
Fin del bucle

Para i desde 1 hasta medio
Para j desde medio hasta n - 1
x = (i + j - 1) % medio + medio + 1
tabla[i][j] = x
tabla[x][j] = i
Fin del bucle
Fin del bucle
Fin

El algoritmo funciona del siguiente modo. Comienza en el método "torneoRecursivoCaso1" para un valor de n dado siendo este potencia de 2. Si n != 2, realizamos llamadas recursivas sucesivamente al método pasando como parámetro n = n/2 y la matriz de enteros hasta que llegamos al caso base. Para cada llamada recursiva en la que n != 2 (no es caso base), una vez que se ha completado la llamada anterior llamaríamos posteriormente al método "completarTablaCaso1", pasando como parámetro n y la matriz. Las entradas que se completan en este método son las correspondientes hasta la columna n (distinta en cada llamada, ya que en cada llamada como hemos dicho anteriormente realizamos a su vez una llamada recursiva pasando como parámetro n = n/2).

Fijándonos más detenidamente en el método "completarTablaCaso1", vemos que para cada llamada a este calculamos previamente la variable medio = n / 2 con el objetivo de establecer el límite de los bucles y para utilizarla a la hora de calcular los valores en las distintas posiciones de la tabla. Hemos utilizado 4 bucles anidados 2 a 2 que funcionan de la siguiente manera:

Los primeros dos bucles comienzan desde i = medio + 1 hasta n (filas) y desde j = 1 hasta n/2 - 1 (columnas). Con estos, rellenamos desde la columna j = 1 hasta la n/2 - 1, las posiciones inferiores correspondientes a dicha columna sumándole n/2 a las posiciones superiores de la misma columna. Esto significa que para cada columna

hasta la fila n, necesitamos los valores de las posiciones superiores para establecer los valores de las posiciones inferiores, y esas posiciones se calculan en las llamadas recursivas anteriores. Por ello, concluímos en que rellenan el cuadrante inferior izquierdo en cada llamada hasta la fila n.

Los últimos dos bucles comienzan desde i = 1 hasta medio (filas) y desde j = medio hasta n - 1. En cada llamada, se encargan respectivamente de calcular los valores de los cuadrantes superiores derecho e inferiores derecho hasta n.

Como conclusión, tomamos en que para cada llamada al método “completarTablaCaso1”, el cuadrante superior izquierdo se encarga la llamada recursiva anterior de rellenarlo para que los 2 primeros bucles anidados rellenen el cuadrante inferior izquierdo y los 2 últimos rellenen el cuadrante superior derecho y el cuadrante inferior derecho. De este modo, solamente en la primera llamada cuando n toma su valor más grande se rellenará en la tabla completa los cuadrantes inferior izquierdo, superior derecho e inferior derecho mientras que el resto de llamadas recursivas rellenarán el cuadrante superior izquierdo.

2.2. Caso 2.1 y 2.2: n es impar o par, solución al problema en n días o n - 1 días

Para el caso 2.1 y 2.2, hemos definido 2 métodos: el método torneoRecursivoCaso2 que, al igual que en el caso anterior, va a tratar las llamadas recursivas y el caso base; y el método completarTablaCaso2 que va a rellenar los diferentes cuadrantes de la tabla. Su implementación es la siguiente:

Método TorneoRecursivoCaso2

```
Método torneoRecursivoCaso2(tabla, n días)
Si n es igual a 2 entonces //caso base
tabla[1][1] = 2
tabla[2][1] = 1
Sino si n modulo 2 no es igual a 0 entonces//descansos para el caso impar
torneoRecursivoCaso2(tabla, n + 1)
Para i desde 1 hasta n
Para j desde 1 hasta n
Si tabla[i][j] es igual a n + 1 entonces
tabla[i][j] = 0
Fin del si
Fin del bucle
Fin del bucle
Sino
completarTablaCaso2(tabla, n)
Fin del si
```

Fin del método

Método CompletarTablaCaso2

Método privado completarTablaCaso2(tabla, n días)
m = n / 2
torneoRecursivoCaso2(tabla, m)

Si m % 2 == 0 entonces **//caso PAR**
Para i desde m + 1 hasta n
Para j desde 1 hasta m - 1
tabla[i][j] = tabla[i - m][j] + m
Fin del bucle
Fin del bucle

Para i desde 1 hasta m
Para j desde m hasta n - 1
Si (i + j) <= n entonces
tabla[i][j] = i + j
Sino
tabla[i][j] = i + j - m
Fin del si
Fin del bucle
Fin del bucle

Para i desde m + 1 hasta n
Para j desde m hasta n - 1
Si i > j entonces
tabla[i][j] = i - j
Sino
tabla[i][j] = (i + m) - j
Fin del si
Fin del bucle
Fin del bucle
Sino **//caso IMPAR**
Para i desde m + 1 hasta n
Para j desde 1 hasta m
Si tabla[i - m][j] == 0 entonces
tabla[i][j] = 0
Sino
tabla[i][j] = tabla[i - m][j] + m
Fin del si
Fin del bucle
Fin del bucle
Para i desde 1 hasta m

```
Para j desde 1 hasta m
Si tabla[i][j] == 0 entonces
tabla[i][j] = i + m
tabla[i + m][j] = i
Fin del si
Fin del bucle
Fin del bucle

Para i desde 1 hasta m
Para j desde m + 1 hasta n - 1
Si (i + j) <= n entonces
tabla[i][j] = i + j
Sino
tabla[i][j] = i + j - m
Fin del si
Fin del bucle
Fin del bucle

Para i desde m + 1 hasta n
Para j desde m + 1 hasta n - 1
Si i > j entonces
tabla[i][j] = i - j
Sino
tabla[i][j] = (i + m) - j
Fin del si
Fin del bucle
Fin del bucle
Fin del si
Fin del método
```

El algoritmo funciona del siguiente modo. Al igual que en el caso 1, las llamadas recursivas las va a tratar el método “torneoRecursivoCaso2”. Para este caso, tenemos 3 condiciones iniciales: si n = 2 (caso base) completamos las posiciones correspondientes, las cuales serían al igual que en el caso 1 y caso 3 las posiciones (1,1) y (2,1). La segunda condición es si n % 2 != 0, la cual calcula para el caso impar los días en los que cada jugador de la tabla debe descansar ya que, como sabemos, para este caso cada jugador debe descansar un día al ser impares. Lo primero que se hace en esta condición es una llamada recursiva para n + 1 (para completar los cuadrantes de la tabla y entrar en la tercera condición en la siguiente llamada) y, posteriormente, mediante 2 bucles anidados que van desde i hasta n (filas) y j hasta n (columnas) y que para cada día calcula el jugador que no jugará dicho día.

Por último, la tercera condición se encarga de llamar en cada llamada recursiva al método “completarTablaCaso2” que rellena los diferentes cuadrantes de la tabla. El primer cuadrante se rellena en este método mediante las llamadas a “torneoRecursivoCaso2” que se realizan posteriormente al cálculo de m = n / 2, pasándole como parámetro la tabla y m. Todas estas llamadas en este método son las que van a rellenar el primer cuadrante de la tabla, al ir reduciendo el valor de n a su mitad, mientras que el resto de cuadrantes los rellenará la primera llamada que realizamos al inicio del algoritmo, para el caso par directamente (ya que pasa por la 3 condición del método “torneoRecursivoCaso2”) y para el caso impar tras la llamada recursiva en la segunda condición estableciendo a n = n + 1 para entrar en la tercera condición del método. Como m = n / 2 puede ser par o impar, el método maneja ambos casos e itera, para el caso en el que m sea par o impar, de distintas maneras, ya que los límites establecidos no son los mismos para ambos casos. De este modo, para m par hemos necesitado de 6 bucles anidados 2 a 2 para y para m impar de 8 bucles anidados 2 a 2.

2.3. Caso 3: n es potencia de 2 para n jugadores

Al igual que en los casos anteriores, hemos definido para el caso 3 dos métodos: torneoRecursivoCaso3 y completarTablaCaso3. Los métodos son los siguientes:

Método TorneoRecursivoCaso3

Método torneoRecursivoCaso3(tabla, n días)
Si n es igual a 2 entonces **//caso base**
tabla[1][1] = 2
tabla[1][2] = 1
Sino
torneoRecursivoCaso3(tabla, n/2)
completarTablaCaso3(tabla, n)
Fin del si
Fin del método

Método CompletarTablaCaso3

Método privado completarTablaCaso3(tabla, n días)
m = n/2
Para i desde 1 hasta m - 1
Para jug desde m + 1 hasta n
tabla[i][jug] = tabla[i][jug - m] + m
Fin del bucle
Fin del bucle

Para dia desde m hasta n - 1
Para jug desde 1 hasta m
x = (jug + dia - 1) % m + m + 1

tabla[dia][x] = jug
tabla[dia][jug] = x
Fin del bucle
Fin del bucle
Fin de la método

El algoritmo funciona de manera similar al caso 1: las llamadas recursivas, al igual que en los 2 casos anteriores, las va a tratar el método “torneoRecursivoCaso3”. Su funcionamiento es el mismo que para el caso1.

Sin embargo, en el método “completarTablaCaso3” nos encontramos con varios cambios:

Como para el caso 1 tratábamos el problema para n - 1 días y para este caso lo tratamos para n jugadores (recordamos que para este caso, las columnas son los jugadores y las filas los días), seguimos completando los cuadrantes mediante 4 bucles anidados 2 a 2, solo que esta vez modificando los límites y los índices, ya que no estamos ante el mismo problema. Los primeros dos bucles van ahora desde i = 1 hasta m - 1 (filas) y desde jug = m + 1 hasta n (columnas) y, al igual que en el caso 1, van a completar el cuadrante inferior izquierdo. Por último, los dos últimos cuadrantes van a ir desde dia = m hasta n (filas) y desde jug = 1 hasta m (columnas), siendo su funcionamiento también el mismo que en el caso 1 calculando los cuadrantes superior derecho e inferior derecho en cada llamada. De este modo, la tabla nos quedaría como el caso 1 pero para las columnas = n - 1 jugadores y para las filas = n días.

3.Estudio teórico

Al tratarse de algoritmos recursivos, vamos a hacer uso del Teorema Maestro en todos nuestros casos, el cual dice así:

$$t(n) = \begin{cases} h(n) & Caso\ base \\ a \cdot t\left(\frac{n}{b}\right) + g(n) & Casos\ recursivos \end{cases}$$

3.1. Caso 1: n es potencia de dos

En este caso identificamos:

- (a=1), hay una sola llamada recursiva por iteración del algoritmo.
- (b=2), porque el tamaño del problema se divide en dos en cada llamada recursiva

- g(n), es el coste de la función “completarTablacaso1” que está dominado por los dos bucles for anidados. Cada uno de estos bloques, recorre hasta n elementos, lo que resulta en un total de hasta $n \times n = n^2$ operaciones. Por lo tanto la complejidad de tiempo de la función es $O(n^2)$.

Para identificar en qué caso del Teorema Maestro nos encontramos, sabemos que $C = 2$, para así ver cual es el resultado de $C = log_b a$. En nuestro caso, resolvemos que $log_b a = log_2 1 = 0$. Como $2 > 0$ llegamos a la conclusión que $C > log_b a$. Identificamos que nos encontramos en el caso 3 del Teorema Maestro, lo cual dictamina que $t(n) = O(f(n)) = O(n^2)$, lo que significa que el tiempo de ejecución aumenta cuadráticamente conforme se aumenta el tamaño del problema

3.2. Caso 2.1: n es impar, solución al problema en n días

En este caso identificamos:

- (a=2), hay dos llamadas recursivas por iteración del algoritmo.
- (b=2), porque el tamaño del problema se divide en dos en cada llamada recursiva
- g(n), es el coste de la parte no recursiva del algoritmo, en este caso ya que se trata de un método (completarTablaCaso2) en el que hay bloques de dos bucles for anidados, es $O(n^2)$.

Para identificar en qué caso del Teorema Maestro nos encontramos, identificamos que $C = 2$, para así ver cual es el resultado de $C = log_b a$. En nuestro caso, resolvemos que $log_b a = log_2 2 = 1$. Como $2 > 1$ llegamos a la conclusión que $C > log_b a$. Identificamos que nos encontramos en el caso 3 del Teorema Maestro, lo cual dictamina que $t(n) = O(f(n)) = O(n^2)$.

3.3. Caso 2.2: n es par, solución al problema en n-1 días

En este caso identificamos:

- (a=2), hay dos llamadas recursivas por iteración del algoritmo.
- (b=2), porque el tamaño del problema se divide en dos en cada llamada recursiva
- g(n), es el coste de la parte no recursiva del algoritmo, en este caso ya que se trata de un método (completarTablaCaso2) en el que hay bloques de dos bucles for anidados, es $O(n^2)$.

Para identificar en qué caso del Teorema Maestro nos encontramos, identificamos que $C = 2$, para así ver cual es el resultado de $C = log_b a$. En nuestro caso, resolvemos que $log_b a = log_2 2 = 1$. Como $2 > 1$ llegamos a la conclusión que $C > log_b a$. Identificamos que nos encontramos en el caso 3 del Teorema Maestro, lo cual dictamina que $t(n) = O(f(n)) = O\left(n^2\right)$.

3.4. Caso 3: n es potencia de 2 para n - 1 jugadores

En este caso identificamos:

- (a=1), hay una sola llamada recursiva por iteración del algoritmo.
- (b=2), porque el tamaño del problema se divide en dos en cada llamada recursiva
- g(n), es el coste de la función “completarTablacaso3” que está dominado por los dos bucles for anidados. Cada uno de estos bloques, recorre hasta n elementos, lo que resulta en un total de hasta $n \times n = n^2$ operaciones. Por lo tanto la complejidad de tiempo de la función es $O\left(n^2\right)$.

Para identificar en que caso del Teorema Maestro nos encontramos, identificamos que $C = 2$, para así ver cual es el resultado de $C = log_b a$. En nuestro caso, resolvemos que $log_b a = log_2 1 = 0$. Como $2 > 0$ llegamos a la conclusión que $C > log_b a$. Identificamos que nos encontramos en el caso 3 del Teorema Maestro, lo cual dictamina que $t(n) = O(f(n)) = O\left(n^2\right)$.

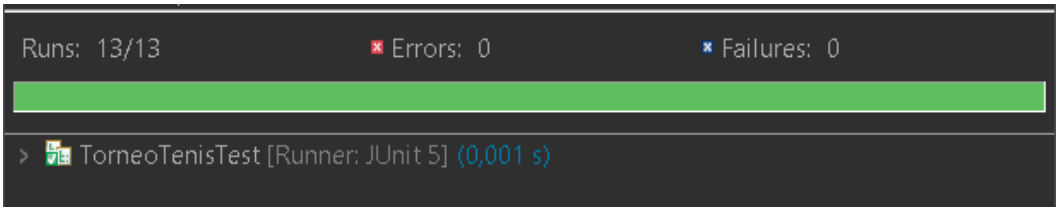
4.Estudio experimental

En el estudio experimental vamos a validar los algoritmos que hemos implementado, realizando las pruebas necesarias, analizando los tiempos de ejecución de cada algoritmo y mostrando dichos resultados en gráficas.

Para saber si las muestras son correctas se usará el estadístico R^2, si está entre 0 y 1 significa que la toma de muestras ha sido correcta. Para ello, hemos creado una carpeta **src/test/java,** que contiene el paquete **org.eda2.practica01** donde se ha creado la clase TorneoTenisTest.java, que la hemos empleado para realizar las pruebas de cada algoritmo y comprobar su correcto funcionamiento comparando el resultado proporcionado para cada valor de n con las tablas de ejemplo del guión de esta práctica. También hemos creado una clase para medir los tiempos, llamada PruebaRendimiento.java, donde se ejecutará cada algoritmo implementado 10

veces y realizará la media, cada uno por separado, copiando el tiempo medio de cada algoritmo en un documento de texto llamado tiempos.txt.

Para el estudio de este apartado, con la clase TorneoTenisTest.java, hemos creado distintas tablas para su comprobación formando los respectivos test en función de las tablas descritas en el guión de la práctica y como se puede ver salen todos en verde:



A continuación exponemos las tablas que hemos utilizado para comprobar el correcto funcionamiento de cada algoritmo:

Para el caso 1:

0	0
0	2
0	1

0	0	0	0
0	2	3	4
0	1	4	3
0	4	1	2
0	3	2	1

0	0	0	0	0	0	0	0
0	2	3	4	5	6	7	8
0	1	4	3	6	7	8	5
0	4	1	2	7	8	5	6
0	3	2	1	8	5	6	7
0	6	7	8	1	4	3	2
0	5	8	7	2	1	4	3
0	8	5	6	3	2	1	4
0	7	6	5	4	3	2	1

Así hasta para n = 64, donde la fila 0 y la columna 0 no se tienen en cuenta.

Para el caso 2 impar:

0	0	0	0	0	0	0	0	0	0
0	2	3	4	5	6	7	8	9	0
0	1	5	3	7	4	8	9	0	6
0	8	1	2	4	5	9	0	6	7
0	5	9	1	3	2	0	6	7	8
0	4	2	0	1	3	6	7	8	9
0	7	8	9	0	1	5	4	3	2
0	6	0	8	2	9	1	5	4	3
0	3	6	7	9	0	2	1	5	4
0	0	4	6	8	7	3	2	1	5

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	2	3	4	5	6	7	8	9	10	11	12	13	0
0	1	4	3	6	7	9	5	10	11	12	13	0	8
0	4	1	2	7	10	5	6	11	12	13	0	8	9
0	3	2	1	11	5	6	7	12	13	0	8	9	10
0	6	7	12	1	4	3	2	13	0	8	9	10	11
0	5	13	7	2	1	4	3	0	8	9	10	11	12
0	0	5	6	3	2	1	4	8	9	10	11	12	13
0	9	10	11	12	13	0	1	7	6	5	4	3	2
0	8	11	10	13	0	2	12	1	7	6	5	4	3
0	11	8	9	0	3	12	13	2	1	7	6	5	4
0	10	9	8	4	12	13	0	3	2	1	7	6	5
0	13	0	5	8	11	10	9	4	3	2	1	7	6
0	12	6	0	9	8	11	10	5	4	3	2	1	7

Donde hemos llegado a comprobar hasta n = 31.

Para el caso 2 par:

0	0	0	0
0	2	3	4
0	1	4	3
0	4	1	2
0	3	2	1

0	0	0	0	0	0
0	2	3	4	5	6
0	1	4	3	6	4
0	4	1	2	4	5
0	3	2	1	3	2
0	4	2	6	1	3
0	3	4	5	2	1

En este caso la comprobación del test ha sido hasta n = 6.

Para el caso 3:

0	0	0	0	0
0	2	1	4	3
0	3	4	1	2
0	4	3	2	1

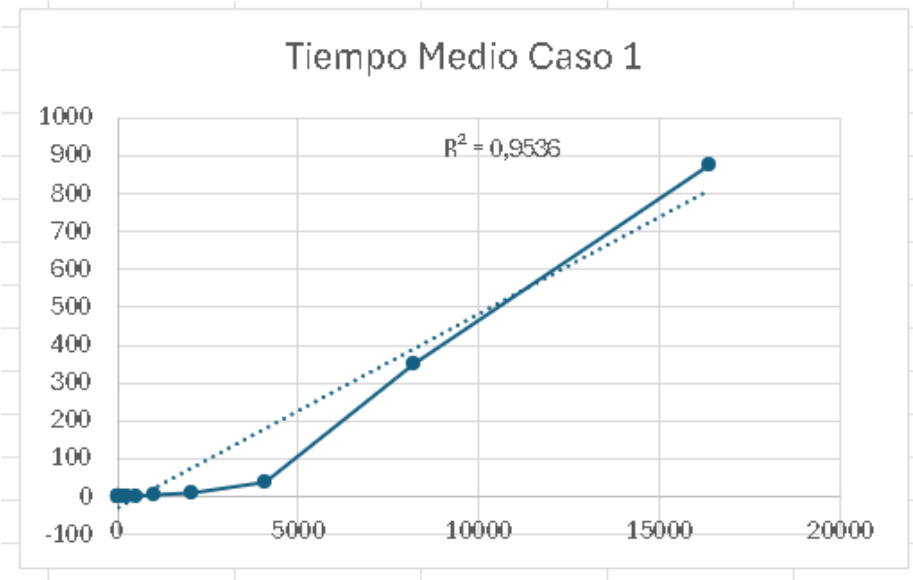
0	0	0	0	0	0	0	0	0
0	2	1	4	3	6	5	8	7
0	3	4	1	2	7	8	5	6
0	4	3	2	1	8	7	6	5
0	5	6	7	8	1	2	3	4
0	6	7	8	5	4	1	2	3
0	7	8	5	6	3	4	1	2
0	8	5	6	7	2	3	4	1

Por último, para el caso 3 ha sido de n = 16.

A continuación, para contrastar con los resultados que hemos obtenido de cada caso en la parte del estudio teórico, hemos medido el tiempo de ejecución para cada caso para diferentes valores de n para comprobar su eficiencia:

4.1. Caso 1: n es potencia de dos

Para el algoritmo implementado en el caso 1, se ha usado un valor máximo para n de 20000, y al ser este potencia de 2, usamos los números **(2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384),** lo cual lo hemos implementado con un bucle que multiplica en cada iteración el valor de n por 2. Al ejecutar la prueba de rendimiento de este algoritmo, el tiempo medio de ejecución es el siguiente:



Como se puede observar la toma de muestras es correcta ya que el estadístico es superior a 0,95, lo que nos dice que es un algoritmo eficiente.

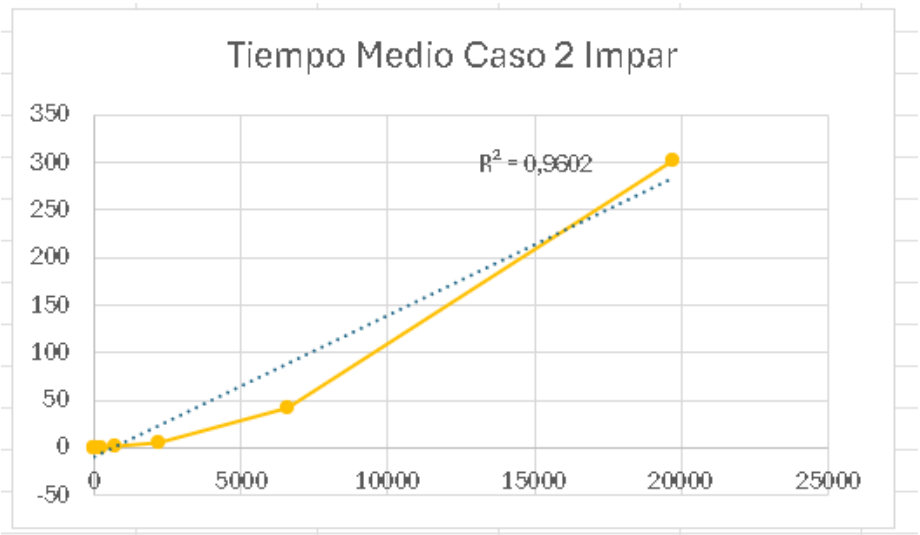
Como ejemplo adicional, mostramos la salida por pantalla esperada de la tabla correspondiente para este caso, dándole un valor simbólico de n = 16:

```
Introduce el caso en el que quieres trabajar (1, 2 ó 3): 1
Introduce nº de filas o nº de jugadores (solo potencia de 2): 16

     0  1  2  3  4  5  6  7  8  9  10  11  12  13  14  15
   -------------------------------------------------
 0 | 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
 1 | 0  2  3  4  5  6  7  8  9  10  11  12  13  14  15  16
 2 | 0  1  4  3  6  7  8  5  10  11  12  13  14  15  16  9
 3 | 0  4  1  2  7  8  5  6  11  12  13  14  15  16  9  10
 4 | 0  3  2  1  8  5  6  7  12  13  14  15  16  9  10  11
 5 | 0  6  7  8  1  4  3  2  13  14  15  16  9  10  11  12
 6 | 0  5  8  7  2  1  4  3  14  15  16  9  10  11  12  13
 7 | 0  8  5  6  3  2  1  4  15  16  9  10  11  12  13  14
 8 | 0  7  6  5  4  3  2  1  16  9  10  11  12  13  14  15
 9 | 0  10  11  12  13  14  15  16  1  8  7  6  5  4  3  2
 10 | 0  9  12  11  14  15  16  13  2  1  8  7  6  5  4  3
 11 | 0  12  9  10  15  16  13  14  3  2  1  8  7  6  5  4
 12 | 0  11  10  9  16  13  14  15  4  3  2  1  8  7  6  5
 13 | 0  14  15  16  9  12  11  10  5  4  3  2  1  8  7  6
 14 | 0  13  16  15  10  9  12  11  6  5  4  3  2  1  8  7
 15 | 0  16  13  14  11  10  9  12  7  6  5  4  3  2  1  8
 16 | 0  15  14  13  12  11  10  9  8  7  6  5  4  3  2  1
```

4.2. Caso 2.1: n es impar, solución al problema en n días

Para el algoritmo caso 2.1 (impar), se ha comprobado hasta un valor de n de 20000 al igual que en el caso 1, y al ser impar, usamos los números **(3, 9, 27, 81, 243, 729, 2187, 6561, 19683),** está vez multiplicando desde n = 3 su valor por 3. Los resultado al ejecutar el algoritmo, con un tiempo medio de rendimiento es:



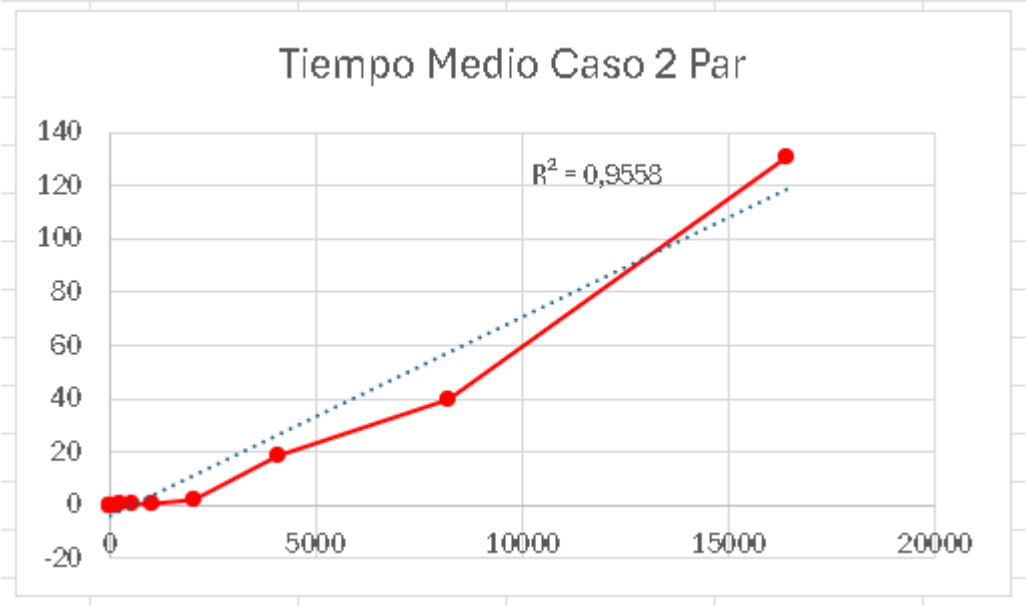
Como se puede observar la toma de muestras es correcta ya que el estadístico es superior a 0,95.
Al igual que en el caso anterior mostramos para este caso otro ejemplo de la tabla esperada para el caso 2 impar, dándole un valor de n = 9:

```
Introduce el caso en el que quieres trabajar (1, 2 ó 3): 2
Introduce nº de dias o columnas: 9

     0  1  2  3  4  5  6  7  8  9
   ------------------------------
 0 | 0  0  0  0  0  0  0  0  0  0
 1 | 0  2  3  4  5  6  7  8  9  0
 2 | 0  1  5  3  7  4  8  9  0  6
 3 | 0  8  1  2  4  5  9  0  6  7
 4 | 0  5  9  1  3  2  0  6  7  8
 5 | 0  4  2  0  1  3  6  7  8  9
 6 | 0  7  8  9  0  1  5  4  3  2
 7 | 0  6  0  8  2  9  1  5  4  3
 8 | 0  3  6  7  9  0  2  1  5  4
 9 | 0  0  4  6  8  7  3  2  1  5
```

4.3. Caso 2.2: n es par, solución al problema en n-1 días

En el caso 2.2, usamos 20000, al ser potencia de 2, usamos los números **(2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384),** como se puede observar en la imagen siguiente el tiempo medio para este algoritmo al ejecutarlo nos sale una muestra correcta y eso se puede justificar porque el estadístico es superior a 0,95.



Para este caso 2 par, de ejemplo, mostramos la tabla esperada para n = 8:

```
Introduce el caso en el que quieres trabajar (1, 2 ó 3): 2
Introduce nº de dias o columnas: 8

     0  1  2  3  4  5  6  7
   -----------------------
 0 | 0  0  0  0  0  0  0  0
 1 | 0  2  3  4  5  6  7  8
 2 | 0  1  4  3  6  7  8  5
 3 | 0  4  1  2  7  8  5  6
 4 | 0  3  2  1  8  5  6  7
 5 | 0  6  7  8  1  4  3  2
 6 | 0  5  8  7  2  1  4  3
 7 | 0  8  5  6  3  2  1  4
 8 | 0  7  6  5  4  3  2  1
```

4.4.Caso 3: n es potencia de 2 para n - 1 jugadores

Por último, para el algoritmo implementado en el caso 3, al ser potencia de 2, usamos los números **(2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384),** al ejecutar la prueba de rendimiento de este algoritmo el tiempo medio de ejecución es el siguiente:



La imagen anterior nos afirma que las muestras son correctas al igual que en los dos casos anteriores, ya que R^2 es superior a 0,95, por lo que el algoritmo es eficiente.

La salida de la tabla esperada para el caso 3, dándole un valor de n = 16 es la siguiente:

```
Introduce el caso en el que quieres trabajar (1, 2 ó 3): 3
Introduce nº de columnas o nº de jugadores (solo potencia de 2): 16

     0   1   2   3   4   5   6   7   8   9   10  11  12  13  14  15  16
   ---------------------------------------------------------------
 0 | 0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
 1 | 0   2   1   4   3   6   5   8   7   10  9   12  11  14  13  16  15
 2 | 0   3   4   1   2   7   8   5   6   11  12  9   10  15  16  13  14
 3 | 0   4   3   2   1   8   7   6   5   12  11  10  9   16  15  14  13
 4 | 0   5   6   7   8   1   2   3   4   13  14  15  16  9   10  11  12
 5 | 0   6   7   8   5   4   1   2   3   14  15  16  13  12  9   10  11
 6 | 0   7   8   5   6   3   4   1   2   15  16  13  14  11  12  9   10
 7 | 0   8   5   6   7   2   3   4   1   16  13  14  15  10  11  12  9
 8 | 0   9   10  11  12  13  14  15  16  1   2   3   4   5   6   7   8
 9 | 0   10  11  12  13  14  15  16  9   8   1   2   3   4   5   6   7
 10 | 0  11  12  13  14  15  16  9   10  7   8   1   2   3   4   5   6
 11 | 0  12  13  14  15  16  9   10  11  6   7   8   1   2   3   4   5
 12 | 0  13  14  15  16  9   10  11  12  5   6   7   8   1   2   3   4
 13 | 0  14  15  16  9   10  11  12  13  4   5   6   7   8   1   2   3
 14 | 0  15  16  9   10  11  12  13  14  3   4   5   6   7   8   1   2
 15 | 0  16  9   10  11  12  13  14  15  2   3   4   5   6   7   8   1
```

Por último, hemos creado una clase llamada MemoriaRendimiento, que se encarga de medir cuantos MB ocupa de la memoria principal las tablas.

```
La tabla ocupa 256,0313 MB de memoria prinicipal para el caso 1
La tabla ocupa 255,9063 MB de memoria prinicipal para el caso 2 par
La tabla ocupa 256,0313 MB de memoria prinicipal para el caso 2 impar
La tabla ocupa 256,0313 MB de memoria prinicipal para el caso 3
```

Como se puede observar para los 3 casos ocupa la misma memoria principal, esto es porque estamos midiendo el valor en megabytes calculando filas x columnas x tamaño de un int en bytes.

Por ello el contenido de las tablas no nos interesa y las implementaciones tampoco, sólo se tiene en cuenta el tamaño de un entero.

PROCEDIMIENTO principal
n = 8192
caso1 = nuevo TorneoTenis con una tabla de tamaño (n + 1) x n
torneoRecursivoCaso1 con la tabla de caso1 y n
mem = calcularMemoria con la tabla de caso1
imprimir "La tabla ocupa " + mem + " MB de memoria principal para el caso 1"

n = 8190
caso2Par = nuevo TorneoTenis con una tabla de tamaño (n + 1) x n
torneoRecursivoCaso2 con la tabla de caso2Par y n
mem = calcularMemoria con la tabla de caso2Par
imprimir "La tabla ocupa " + mem + " MB de memoria principal para el caso 2 par"

n = 8191
caso2Impar = nuevo TorneoTenis con una tabla de tamaño (n + 2) x (n + 1)
torneoRecursivoCaso2 con la tabla de caso2Impar y n
mem = calcularMemoria con la tabla de caso2Impar
imprimir "La tabla ocupa " + mem + " MB de memoria principal para el caso 2 impar"

n = 8192
caso3 = nuevo TorneoTenis con una tabla de tamaño n x (n + 1)
torneoRecursivoCaso3 con la tabla de caso3 y n
mem = calcularMemoria con la tabla de caso3
imprimir "La tabla ocupa " + mem + " MB de memoria principal para el caso 3"
FIN PROCEDIMIENTO

PROCEDIMIENTO calcularMemoria(m)
t = longitud de m * longitud de m[0] * tamaño de Integer / 8.0
conversionAMB = t / 1024^2
devolver conversionAMB

FIN PROCEDIMIENTO

5.Bibliografía

https://es.khanacademy.org/computing/computer-science/algorithms/merge-sort/a/divide-and-conquer-algorithms

https://madi.nekomath.com/P4/TeoremaMaestro.html